```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files Indian Liver…t (ILPD).csv

Indian Liver Patient Dataset (ILPD).csv(text/csv) - 23755 bytes, last modified: 3/7/2026 - 100% done

```
Saving Indian Liver Patient Dataset (ILPD).csv to Indian Liver Patient Dataset (ILPD)
```

```
# Standard column names for the ILPD dataset
columns = [
    'Age', 'Gender', 'Total_Bilirubin', 'Direct_Bilirubin',
    'Alkaline_Phosphotase', 'Alamine_Aminotransferase',
    'Aspartate_Aminotransferase', 'Total_Protiens', 'Albumin',
    'Albumin_and_Globulin_Ratio', 'Target'
]

# Load the dataset
df = pd.read_csv('Indian Liver Patient Dataset (ILPD).csv', names=columns)

# Preview the first few rows
df.head()
```

	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Amin
0	65	Female	0.7	0.1	187	
1	62	Male	10.9	5.5	699	
2	62	Male	7.3	4.1	490	
3	58	Male	1.0	0.4	182	
4	72	Male	3.9	2.0	195	

Next steps: Generate code with df New interactive sheet

CHECKING NUMBER OF ROWS & COLUMNS

```
df.shape
```

```
(583, 11)
```

CHECKING DATA TYPES

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 583 entries, 0 to 582
Data columns (total 11 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Age                         583 non-null    int64
 1   Gender                      583 non-null    object
 2   Total_Bilirubin             583 non-null    float64
 3   Direct_Bilirubin            583 non-null    float64
 4   Alkaline_Phosphotase        583 non-null    int64
 5   Alamine_Aminotransferase    583 non-null    int64
 6   Aspartate_Aminotransferase  583 non-null    int64
 7   Total_Protiens              583 non-null    float64
 8   Albumin                     583 non-null    float64
 9   Albumin_and_Globulin_Ratio  579 non-null    float64
 10  Target                      583 non-null    int64
dtypes: float64(5), int64(5), object(1)
memory usage: 50.2+ KB
```

CLASS DISTRIBUTION

```
df['Target'].value_counts()
```

Target	count
1	416
2	167

dtype: int64

CONVERTING CLASS DISTRIBUTION INTO PERCENTAGE

```
df['Target'].value_counts(normalize=True) * 100
```

Target	proportion
1	71.35506
2	28.64494

dtype: float64

Check categorical columns:

```
df['Gender'].unique()
```

```
array(['Female', 'Male'], dtype=object)
```

CHECK NEGATIVE VALUES

```
num_cols = df.select_dtypes(include=['int64','float64']).columns

for col in num_cols:
    print(col, (df[col] < 0).sum())
```

```
Age 0
Total_Bilirubin 0
Direct_Bilirubin 0
Alkaline_Phosphotase 0
Alamine_Aminotransferase 0
Aspartate_Aminotransferase 0
Total_Protiens 0
Albumin 0
Albumin_and_Globulin_Ratio 0
Target 0
```

Check Missing Values

```
df.isnull().sum()
```

	0
Age	0
Gender	0
Total_Bilirubin	0
Direct_Bilirubin	0
Alkaline_Phosphotase	0
Alamine_Aminotransferase	0
Aspartate_Aminotransferase	0
Total_Protiens	0
Albumin	0
Albumin_and_Globulin_Ratio	4
Target	0

dtype: int64

```
#dropping missing values
df=df.dropna()
```

Check Extreme Values

```
df.describe()
```

	Age	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Ar
count	579.000000	579.000000	579.000000	579.000000	
mean	44.782383	3.315371	1.494128	291.366149	
std	16.221786	6.227716	2.816499	243.561863	
min	4.000000	0.400000	0.100000	63.000000	
25%	33.000000	0.800000	0.200000	175.500000	
50%	45.000000	1.000000	0.300000	208.000000	
75%	58.000000	2.600000	1.300000	298.000000	
max	90.000000	75.000000	19.700000	2110.000000	

DETECT INVALID VALUES

```
print("Age < 0:", len(df[df['Age'] < 0]))
print("Age > 120:", len(df[df['Age'] > 120]))

print("Total_Bilirubin < 0:", len(df[df['Total_Bilirubin'] < 0]))
print("Direct_Bilirubin < 0:", len(df[df['Direct_Bilirubin'] < 0]))

print("Total_Protiens < 0:", len(df[df['Total_Protiens'] < 0]))
print("Albumin < 0:", len(df[df['Albumin'] < 0]))

print("Alkaline_Phosphotase < 0:", len(df[df['Alkaline_Phosphotase'] < 0]))
print("Alamine_Aminotransferase < 0:", len(df[df['Alamine_Aminotransferase'] < 0]))
print("Aspartate_Aminotransferase < 0:", len(df[df['Aspartate_Aminotransferase'] < 0

print("Albumin_and_Globulin_Ratio < 0:", len(df[df['Albumin_and_Globulin_Ratio'] < 0
```

```
Age < 0: 0
Age > 120: 0
Total_Bilirubin < 0: 0
Direct_Bilirubin < 0: 0
Total_Protiens < 0: 0
Albumin < 0: 0
Alkaline_Phosphotase < 0: 0
Alamine_Aminotransferase < 0: 0
Aspartate_Aminotransferase < 0: 0
Albumin_and_Globulin_Ratio < 0: 0
```

```
#checks infinite values in a DataFrame
import numpy as np
np.isinf(df.select_dtypes(include=np.number)).sum()

df['Gender'].unique()

duplicate = df.duplicated().sum()
print(f'duplicate rows: {duplicate}')
```

```
duplicate rows: 13
```

```
#REMOVE dublicate rows
df = df.drop_duplicates()
```

Encode Categorical Variables

```
df['Gender'] = df['Gender'].map({'Male': 1, 'Female': 0})
df['Gender'].head()
```

	Gender
0	0
1	1
2	1
3	1
4	1

dtype: int64

Visualize Class Distribution

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x='Target', data=df)

plt.title("Class Distribution of Target Variable")
plt.xlabel("Target")
plt.ylabel("Count")

plt.show()
```

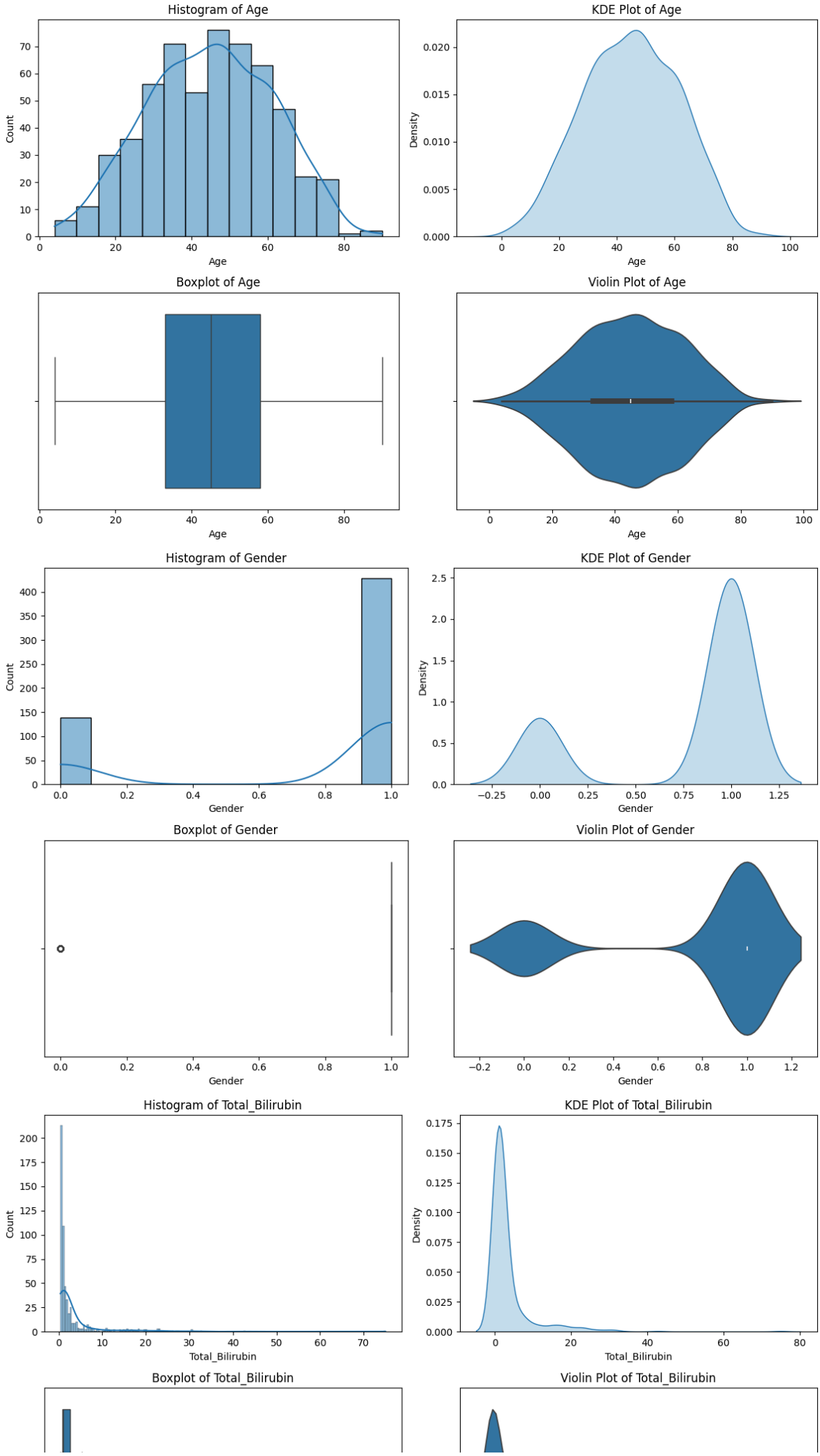
The target variable distribution shows that the dataset is imbalanced, with significantly more samples belonging to class 1 than class 2. Class imbalance can negatively impact model performance because the model may become biased toward the majority class. We will be using CLASS WEIGHT because it give higher importance to minority class during training.

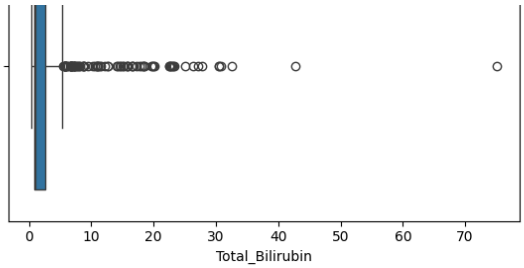
```
**Small dataset → Class weights
Medium dataset → SMOTE
Deep learning → Focal loss**
```

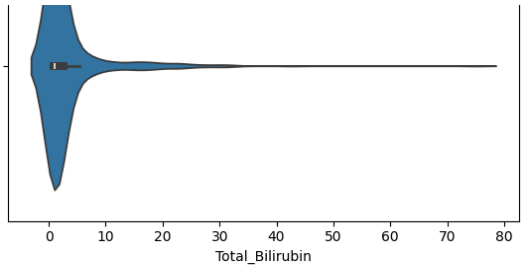
##univariate analysis

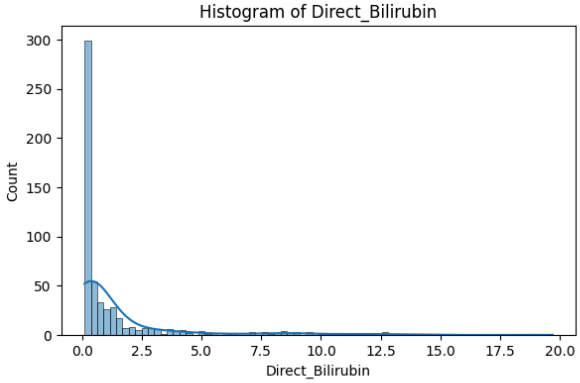
```
import seaborn as sns
import matplotlib.pyplot as plt

# Select numeric columns
num_cols = df.select_dtypes(include=['int64','float64']).columns

for col in num_cols:

    plt.figure(figsize=(12,8))

    # Histogram
    plt.subplot(2,2,1)
    sns.histplot(df[col], kde=True)
    plt.title(f'Histogram of {col}')

    # KDE Plot
```

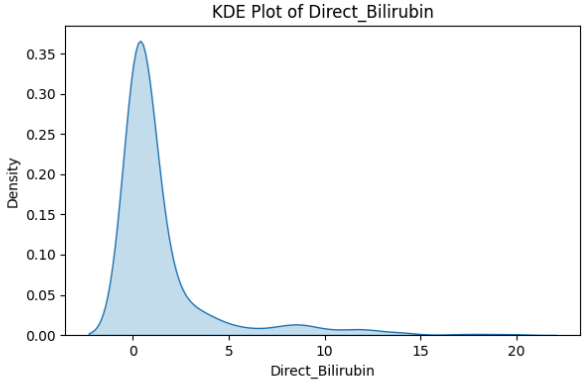
```
        plt.subplot(2,2,2)
        sns.kdeplot(df[col], fill=True)
        plt.title(f'KDE Plot of {col}')

        # Boxplot
        plt.subplot(2,2,3)
        sns.boxplot(x=df[col])
        plt.title(f'Boxplot of {col}')

        # Violin Plot
        plt.subplot(2,2,4)
        sns.violinplot(x=df[col])
        plt.title(f'Violin Plot of {col}')

        plt.tight_layout()
        plt.show()
```

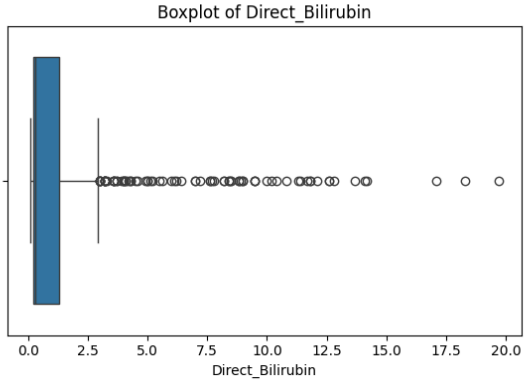

Histogram of Age
Count
Age
KDE Plot of Age
Density
Age
Boxplot of Age
Age
Violin Plot of Age
Age
Histogram of Gender
Count
Gender
KDE Plot of Gender
Density
Gender
Boxplot of Gender
Gender
Violin Plot of Gender
Gender
Histogram of Total_Bilirubin
Count
Total_Bilirubin
KDE Plot of Total_Bilirubin
Density
Total_Bilirubin
Boxplot of Total_Bilirubin
Violin Plot of Total_Bilirubin

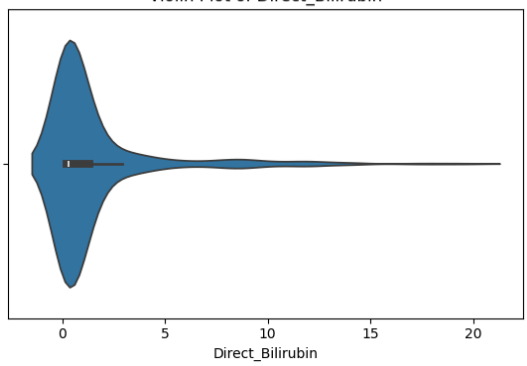

0
10
20
30
40
50
60
70
Total_Bilirubin

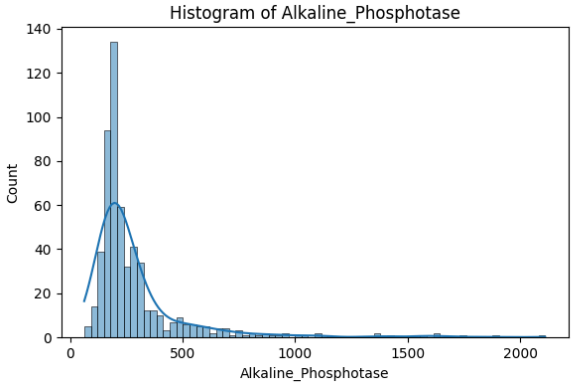

0
10
20
30
40
50
60
70
80
Total_Bilirubin

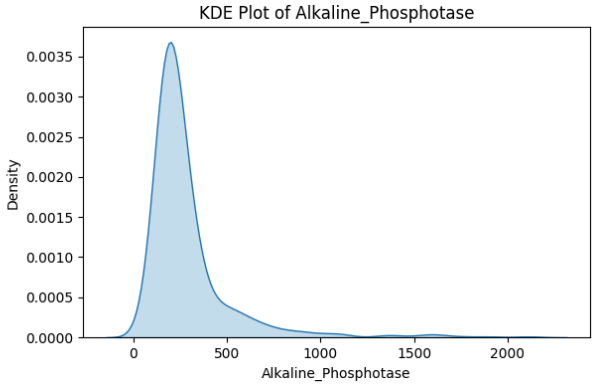

Histogram of Direct_Bilirubin
Count
300
250
200
150
100
50
0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Direct_Bilirubin

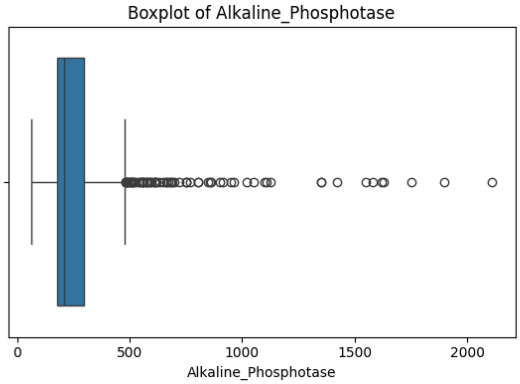

KDE Plot of Direct_Bilirubin
Density
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
5
10
15
20
Direct_Bilirubin

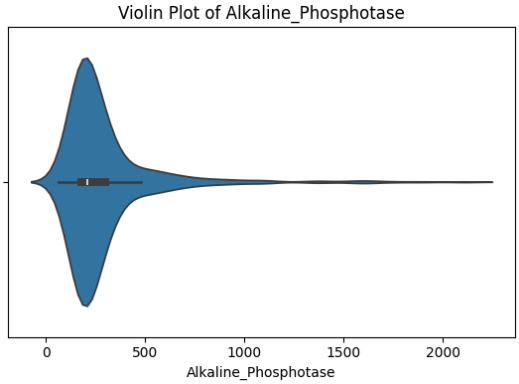

Boxplot of Direct_Bilirubin
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Direct_Bilirubin

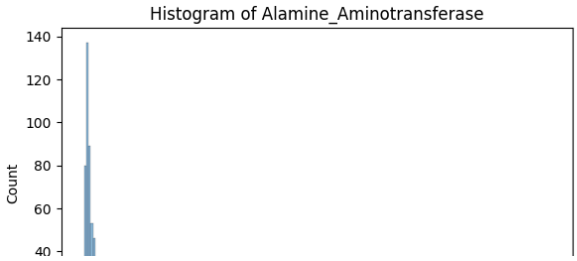

Violin Plot of Direct_Bilirubin
0
5
10
15
20
Direct_Bilirubin

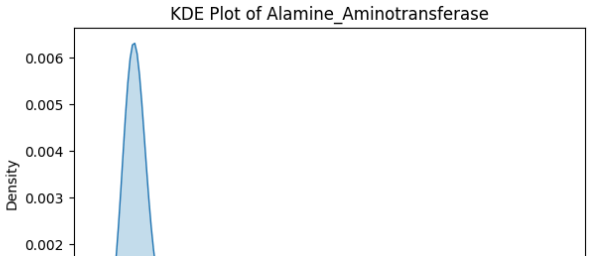

Histogram of Alkaline_Phosphotase
Count
140
120
100
80
60
40
20
0
0
500
1000
1500
2000
Alkaline_Phosphotase

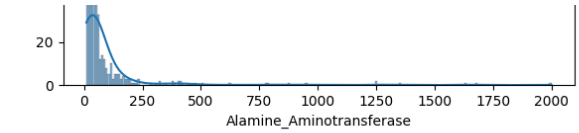

KDE Plot of Alkaline_Phosphotase
Density
0.0035
0.0030
0.0025
0.0020
0.0015
0.0010
0.0005
0.0000
0
500
1000
1500
2000
Alkaline_Phosphotase

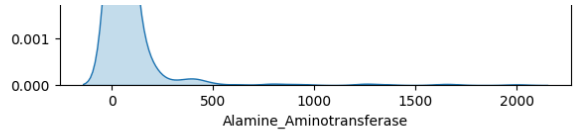

Boxplot of Alkaline_Phosphotase
0
500
1000
1500
2000
Alkaline_Phosphotase


Violin Plot of Alkaline_Phosphotase
0
500
1000
1500
2000
Alkaline_Phosphotase

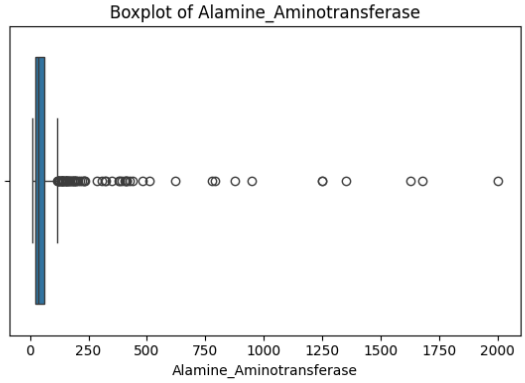

Histogram of Alamine_Aminotransferase
Count
140
120
100
80
60
40


KDE Plot of Alamine_Aminotransferase
Density
0.006
0.005
0.004
0.003
0.002

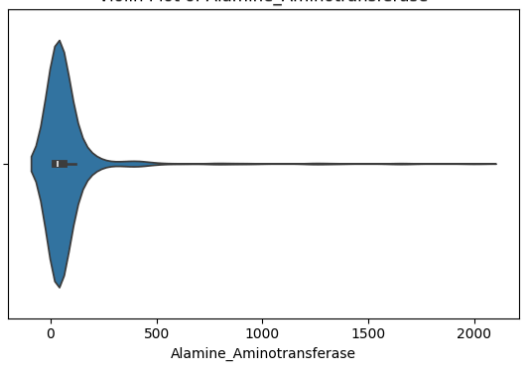




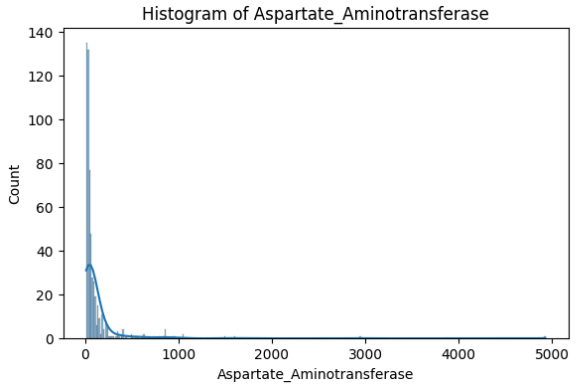
Boxplot of Alamine_Aminotransferase



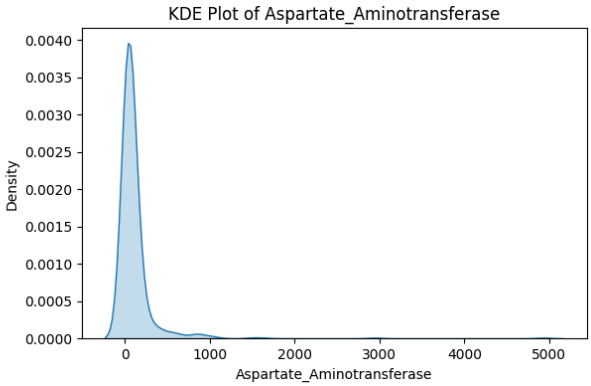
Violin Plot of Alamine_Aminotransferase



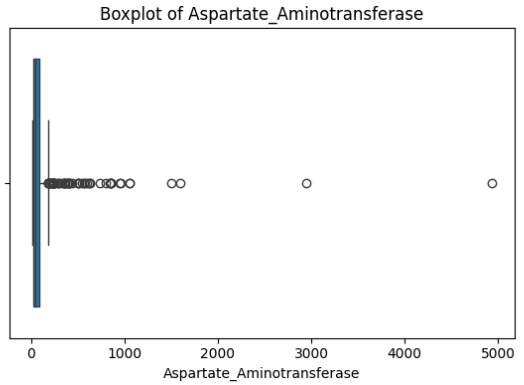
Histogram of Aspartate_Aminotransferase



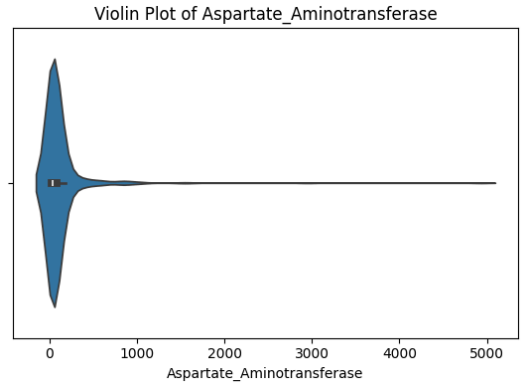
KDE Plot of Aspartate_Aminotransferase



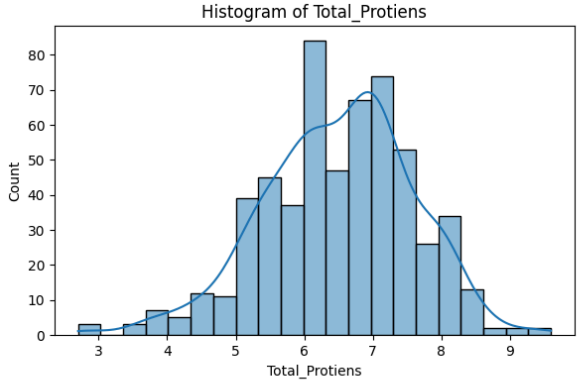
Boxplot of Aspartate_Aminotransferase



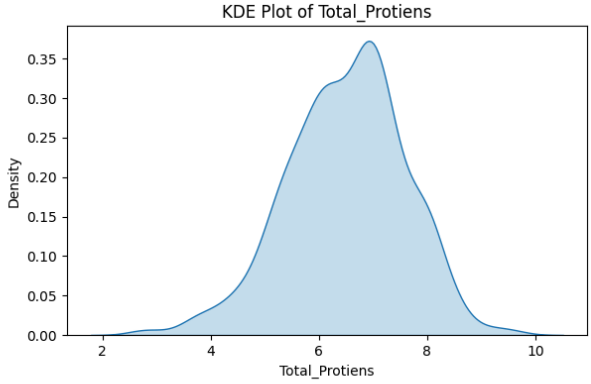
Violin Plot of Aspartate_Aminotransferase



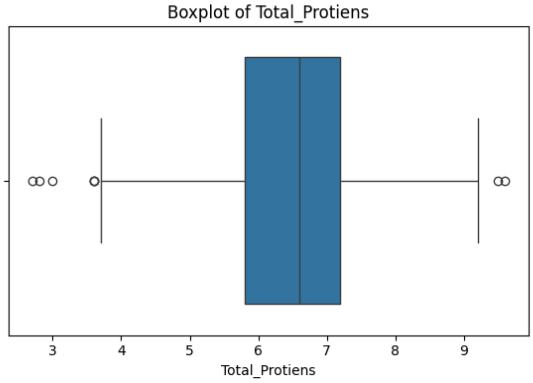
Histogram of Total_Protiens



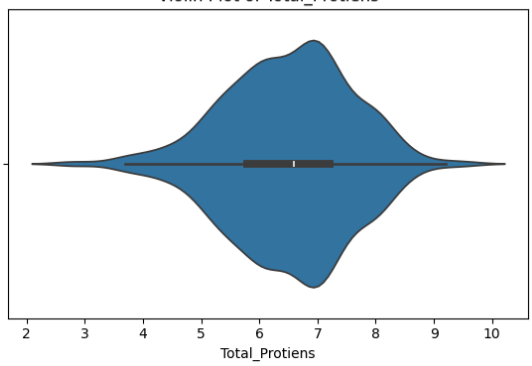
KDE Plot of Total_Protiens

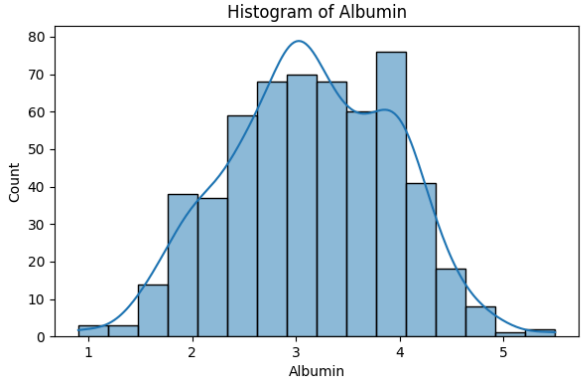


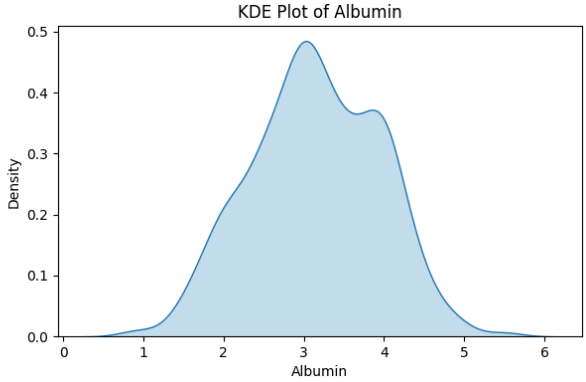
Boxplot of Total_Protiens

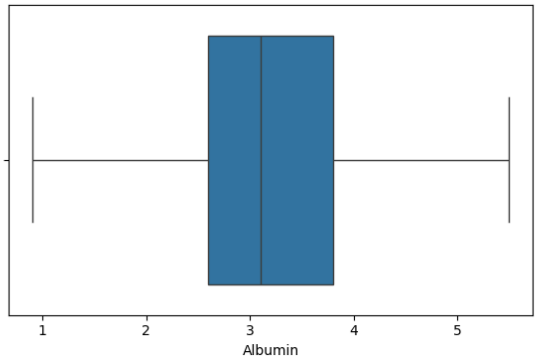


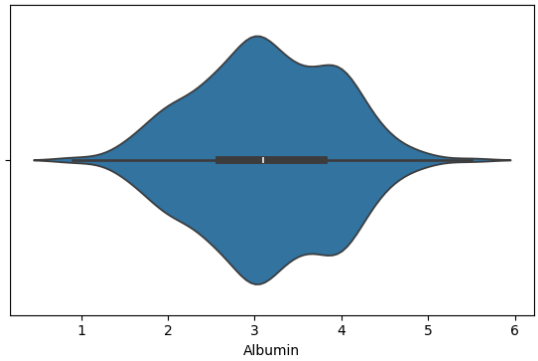
Violin Plot of Total_Protiens

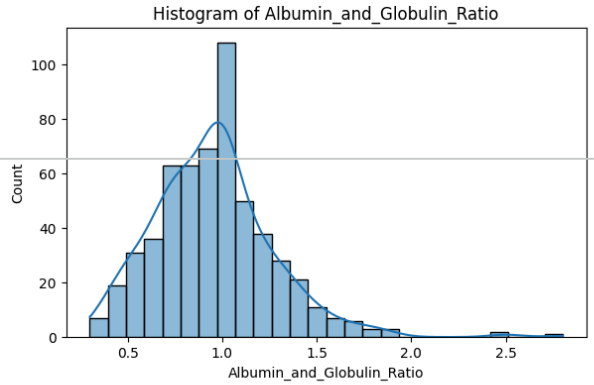


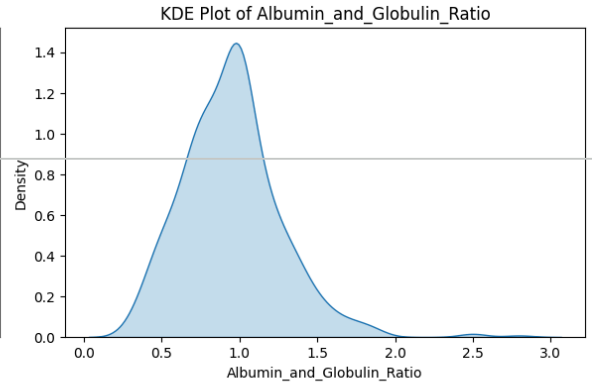

Histogram of Albumin
Count
Albumin

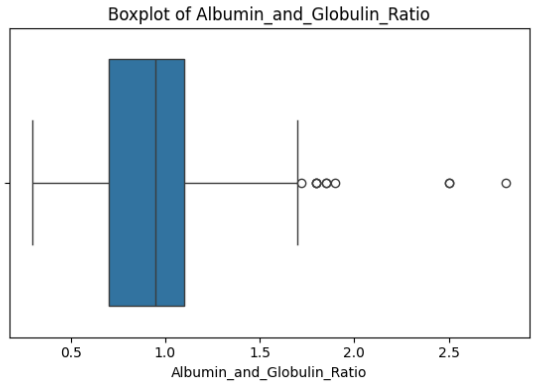

KDE Plot of Albumin
Density
Albumin

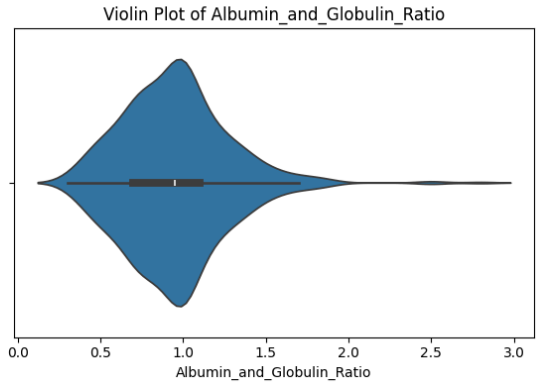

Boxplot of Albumin
Albumin

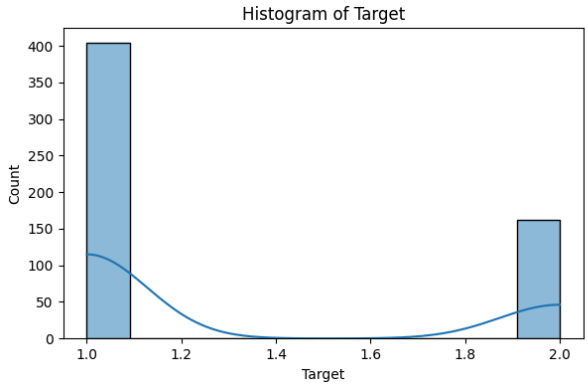

Violin Plot of Albumin
Albumin

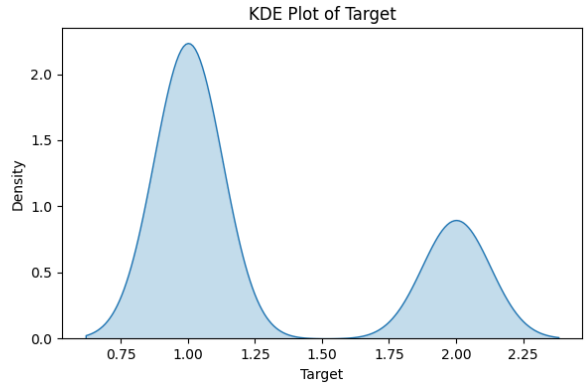

Histogram of Albumin_and_Globulin_Ratio
Count
Albumin_and_Globulin_Ratio

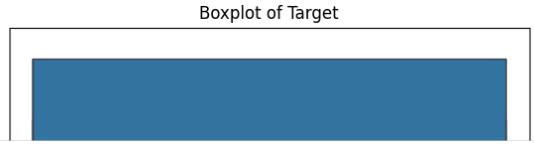

KDE Plot of Albumin_and_Globulin_Ratio
Density
Albumin_and_Globulin_Ratio

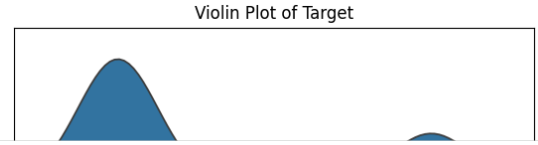

Boxplot of Albumin_and_Globulin_Ratio
Albumin_and_Globulin_Ratio

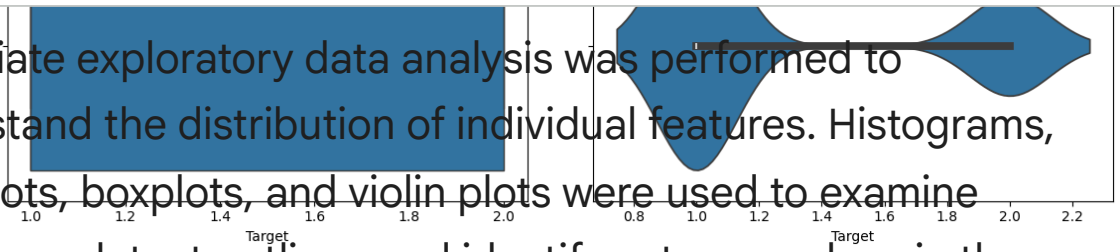

Violin Plot of Albumin_and_Globulin_Ratio
Albumin_and_Globulin_Ratio


Histogram of Target
Count
Target


KDE Plot of Target
Density
Target


Boxplot of Target


Violin Plot of Target



Univariate exploratory data analysis was performed to understand the distribution of individual features. Histograms, KDE plots, boxplots, and violin plots were used to examine skewness, detect outliers, and identify extreme values in the dataset.

```
import pandas as pd

# Select numeric columns (excluding target)
num_cols = df.select_dtypes(include=['int64','float64']).columns.drop('Target')

summary = pd.DataFrame({
    'Mean': df[num_cols].mean(),
    'Median': df[num_cols].median(),
    'Std Dev': df[num_cols].std(),
    'Skewness': df[num_cols].skew(),
    'Kurtosis': df[num_cols].kurt()
})

print(summary)
```

```
                                   Mean  Median     Std Dev   Skewness  \
Age                           44.886926   45.00   16.274893  -0.050998
Gender                         0.756184    1.00    0.429763  -1.196437
Total_Bilirubin                3.338869    1.00    6.286728   4.852444
Direct_Bilirubin               1.505830    0.30    2.841485   3.174186
Alkaline_Phosphotase         292.567138  208.00  245.936559   3.716540
Alamine_Aminotransferase      80.143110   35.00  182.044881   6.672512
Aspartate_Aminotransferase   109.892226   41.00  291.841897  10.524552
Total_Protiens                 6.494876    6.60    1.087512  -0.317937
Albumin                        3.145583    3.10    0.795745  -0.065818
Albumin_and_Globulin_Ratio     0.948004    0.95    0.319635   1.001294

                              Kurtosis
Age                          -0.570584
Gender                       -0.570567
Total_Bilirubin              36.262008
Direct_Bilirubin             11.021222
Alkaline_Phosphotase         17.236024
Alamine_Aminotransferase     52.345206
Aspartate_Aminotransferase  149.305593
Total_Protiens                0.272734
Albumin                      -0.375984
Albumin_and_Globulin_Ratio    3.354530
```

```
Mean ≠ Median   skewed distribution
 High kurtosis   presence of outliers
 Long tails      extreme values
```

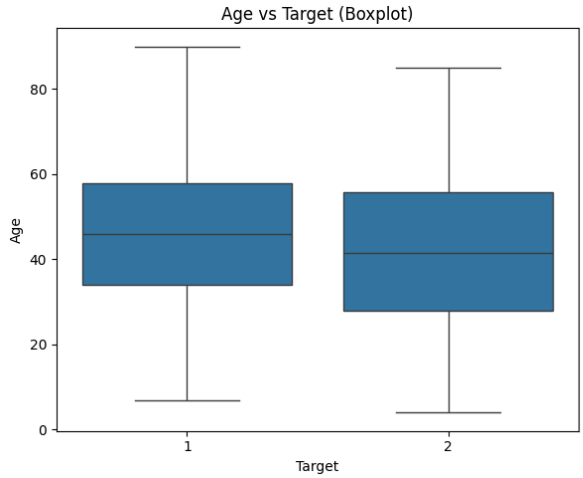
```
for col in num_cols:
    mean = df[col].mean()
    median = df[col].median()
```

```
    kurt = df[col].kurt()

    if mean != median:
        print(col, "may be skewed")

    if kurt > 3:
        print(col, "may contain outliers")
```

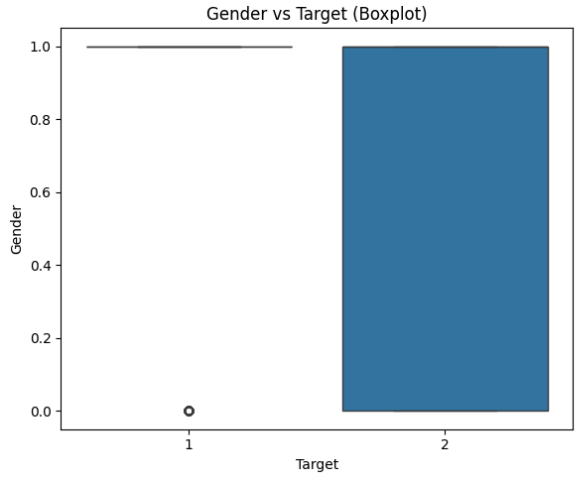
```
Age may be skewed
Gender may be skewed
Total_Bilirubin may be skewed
Total_Bilirubin may contain outliers
Direct_Bilirubin may be skewed
Direct_Bilirubin may contain outliers
Alkaline_Phosphotase may be skewed
Alkaline_Phosphotase may contain outliers
Alamine_Aminotransferase may be skewed
Alamine_Aminotransferase may contain outliers
Aspartate_Aminotransferase may be skewed
Aspartate_Aminotransferase may contain outliers
Total_Protiens may be skewed
Albumin may be skewed
Albumin_and_Globulin_Ratio may be skewed
Albumin_and_Globulin_Ratio may contain outliers
```

Bivariate EDA (Feature vs Target)

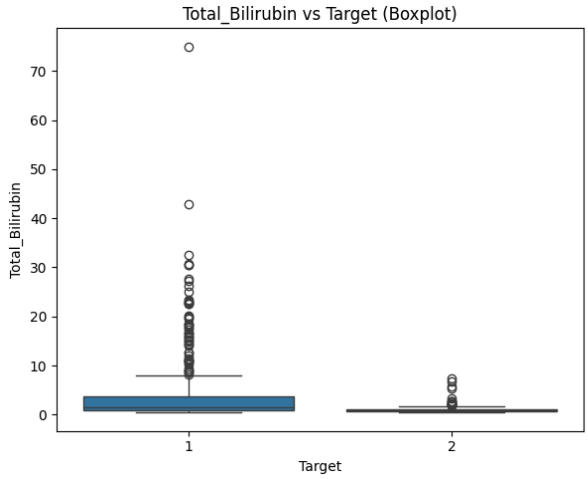
```
# select numeric columns except target
num_cols = df.select_dtypes(include=['int64','float64']).columns.drop('Target')

for col in num_cols:

    plt.figure(figsize=(12,5))

    # Boxplot
    plt.subplot(1,2,1)
    sns.boxplot(x='Target', y=col, data=df)
    plt.title(f'{col} vs Target (Boxplot)')

    # Violinplot
    plt.subplot(1,2,2)
    sns.violinplot(x='Target', y=col, data=df)
    plt.title(f'{col} vs Target (Violin Plot)')

    plt.tight_layout()
    plt.show()
```

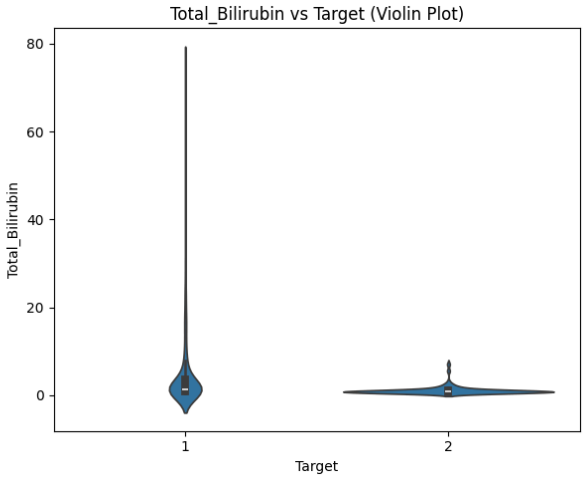

Age vs Target (Boxplot)
Age
Target

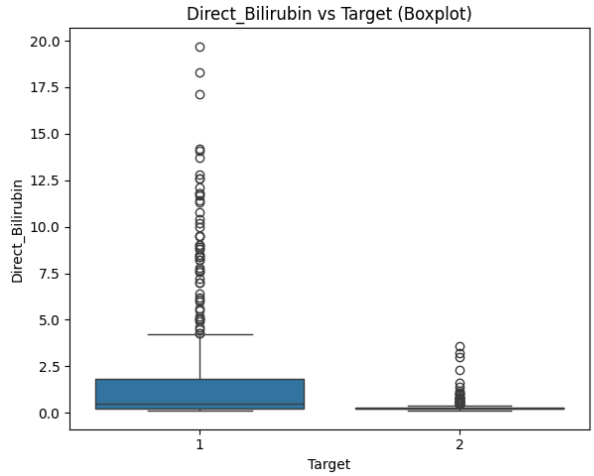
Age vs Target (Violin Plot)
Age
Target

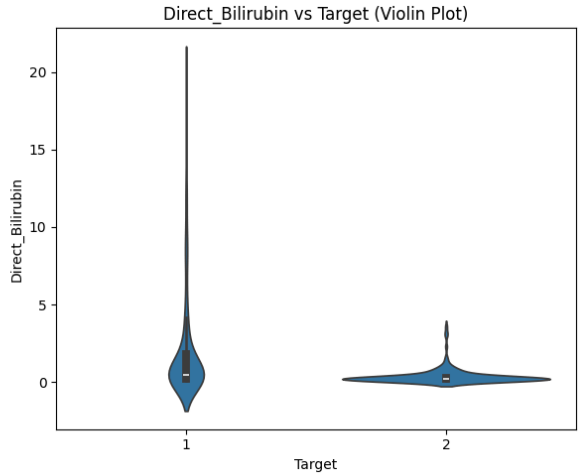

Gender vs Target (Boxplot)
Gender
Target

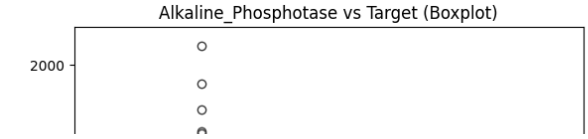
Gender vs Target (Violin Plot)
Gender
Target

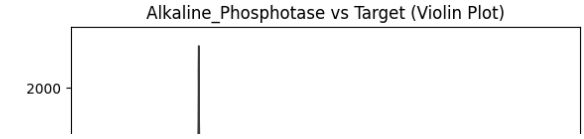

Total_Bilirubin vs Target (Boxplot)
Total_Bilirubin
Target

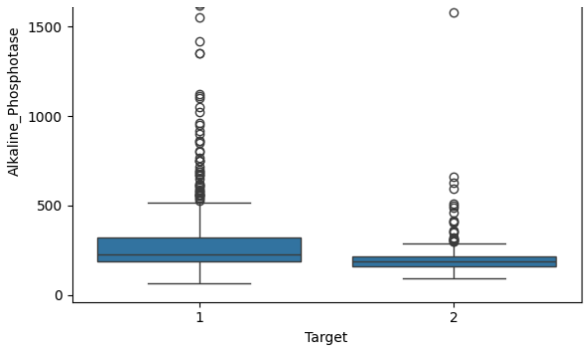

Total_Bilirubin vs Target (Violin Plot)
Total_Bilirubin
Target

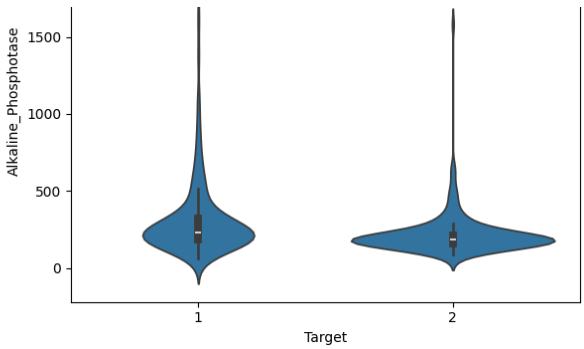

Direct_Bilirubin vs Target (Boxplot)
Direct_Bilirubin
Target


Direct_Bilirubin vs Target (Violin Plot)
Direct_Bilirubin
Target

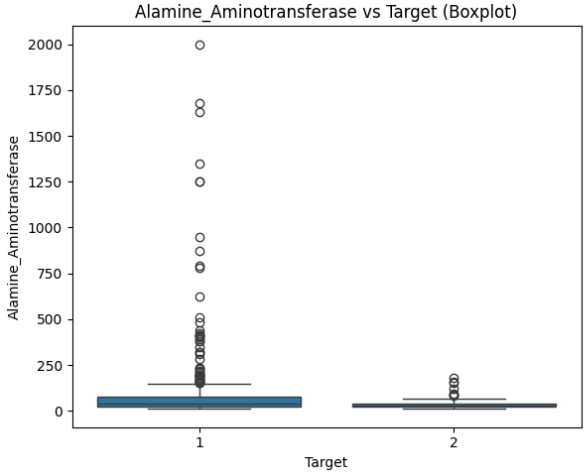

Alkaline_Phosphotase vs Target (Boxplot)


Alkaline_Phosphotase vs Target (Violin Plot)

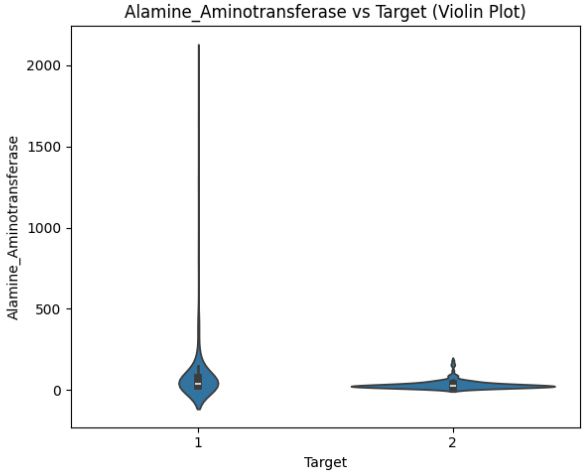




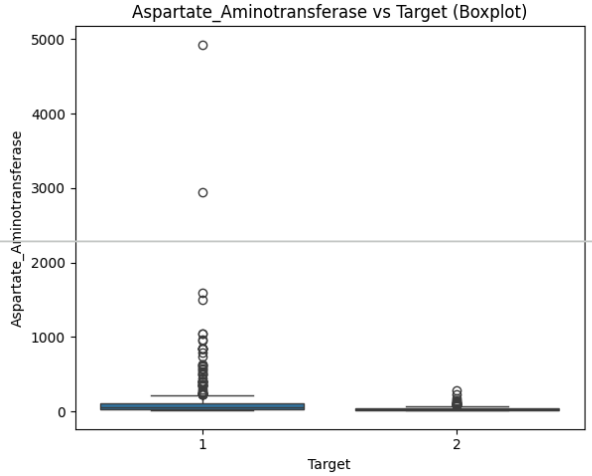
Alamine_Aminotransferase vs Target (Boxplot)



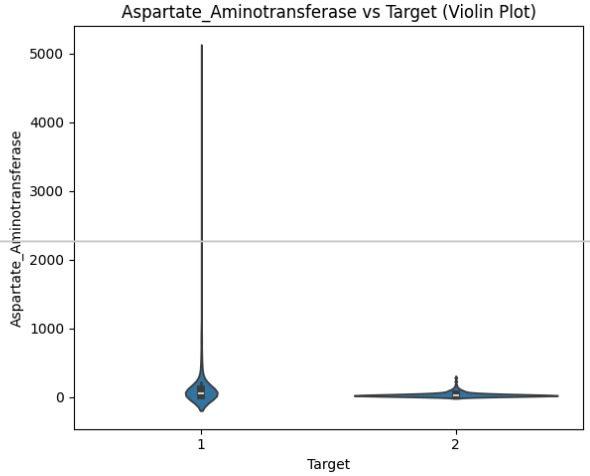
Alamine_Aminotransferase vs Target (Violin Plot)



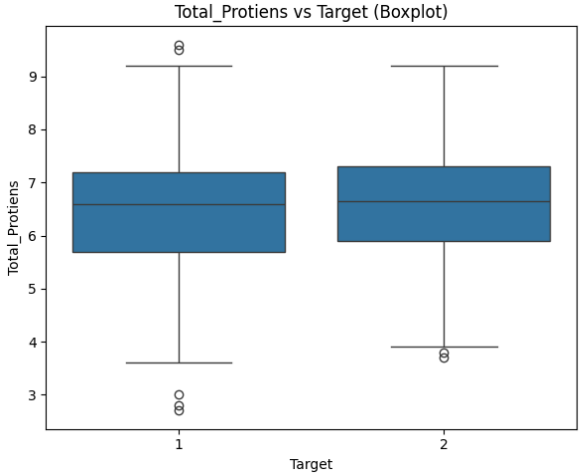
Aspartate_Aminotransferase vs Target (Boxplot)



Aspartate_Aminotransferase vs Target (Violin Plot)

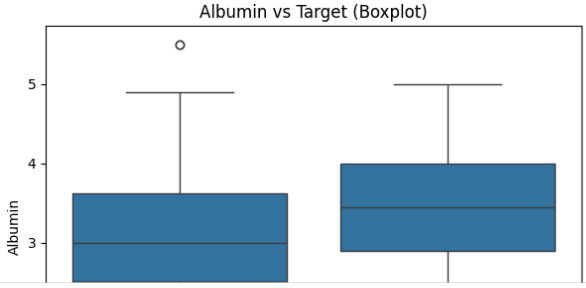
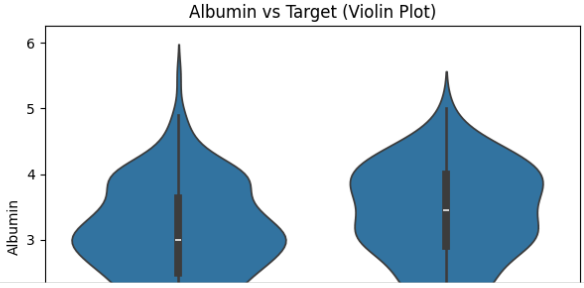


Total_Protiens vs Target (Boxplot)



Total_Protiens vs Target (Violin Plot)

Albumin vs Target (Boxplot)



Albumin vs Target (Violin Plot)